

Custom Wake Word Build Plan: "Sherlock" (LiveKit Edition, v2)

1. Objective

- **Target Phrasing:** Build a single custom wake-word model responding to four variations: "sherlock", "hey sherlock", "hi sherlock", and "hello sherlock".
- **Output Format:** Standard ONNX (.onnx) classifier file.
- **Accuracy Bar:** Achieve a high recall target ($\geq 70\%$ true-positive rate) while keeping the false-positive rate low.
- **v2 addition:** Blend synthetic TTS data with real recordings of your own voice and your own room's background noise, so the model is tuned to how you'll actually use it — not just synthetic speech.

2. Approach: Multi-Phrase Class Training + Personal Voice Data

All four phrasings are grouped into a single model configuration, sharing the acoustic embedding space for the root word "Sherlock." On top of the synthetic TTS data, your own recorded voice clips and room noise are folded in as additional training data before the final training run.

3. Environment Setup

System Dependencies

- **Ubuntu/Debian:** `sudo apt install espeak-ng ffmpeg sox libsndfile1 portaudio19-dev`
- **macOS:** `brew install espeak-ng ffmpeg sox portaudio`
- **Windows:** `libsndfile` and `portaudio` ship bundled with the `soundfile`/`pyaudio` Python wheels — no separate install needed. Install `espeak-ng`, `ffmpeg`, and `sox` manually.

Python Package Installation

```
bash

pip install "livekit-wakeword[train,eval,export]"
```

GPU Verification (RTX 3050)

Training runs locally on your GPU, but `pip` can silently install a CPU-only PyTorch build. Verify before doing anything else:

```
bash
```

```
python -c "import torch; print(torch.cuda.is_available())"
```

- If this prints `True` → you're good, training will use the 3050 automatically.
- If it prints `False` → reinstall PyTorch with a CUDA-matched build (check your installed CUDA driver version with `nvidia-smi` first, then grab the matching wheel from pytorch.org).

Note: with `model_size: small`, the classifier itself is tiny, so the 3050 is more than sufficient — GPU speed mainly helps during the training phases, not the CPU-bound Piper TTS data generation step.

Confirm exact CLI commands for your installed version

Command names have shifted slightly across recent LiveKit-wakeword releases. Before running anything, check:

```
bash
```

```
livekit-wakeword --help
```

```
livekit-wakeword <stage> --help # e.g. livekit-wakeword train --help
```

This avoids a mistyped stage name silently doing the wrong thing.

4. Configuration (`sherlock.yaml`)

```
yaml
```

```
model_name: sherlock
target_phrases:
  - "sherlock"
  - "hey sherlock"
  - "hi sherlock"
  - "hello sherlock"

n_samples: 10000          # Total synthetic clips generated via TTS (library
steps: 50000              # Training steps (library default)
target_fp_per_hour: 0.2   # Library default – not an aggressive custom value

model:
  model_type: conv_attention # Best at separating phonetically similar phrase
  model_size: small         # Options: tiny, small, medium, large

# --- v2 addition: your own room's background noise as negative examples ---
n_background_samples: 200
n_background_samples_val: 40

augmentation:
  background_paths:
    - ./data/backgrounds      # default MUSAN noise (from setup)
    - ./data/my-room-noise    # your own recordings – see Section 5b
```

5. Data Collection

5a. Record your own voice (new step)

Record yourself saying each of the four target phrases, aiming for real variety:

- **Quantity:** 20–50+ clips per phrase minimum (more is better)
- **Distance:** close to the mic, and from across the room
- **Tone:** casual, urgent, questioning, tired — however you'd realistically say it
- **Conditions:** quiet room, and with some background noise (TV, fan, typing)

Save these as individual short `.wav` files in a folder, e.g. `./data/my-voice/`.

Injecting them into training — check the README's full config option list first for a direct "custom positive audio path" field (the library documents an equivalent mechanism for background noise, so a similar one likely exists for positive samples too). If no such field exists, the documented fallback is:

1. Run your `.wav` files through the same embedding-extraction step the pipeline uses on synthetic clips (feature shape: `(N, 16, 96)`).
2. Append the resulting arrays to `positive_features_train.npy` before the `train` stage runs.

This is more manual than a config flag, but it's fully compatible with the pipeline's documented data format.

5b. Record your own room's background noise (new step)

Record 10–15 minutes of your actual room's ambient sound — laptop fan, keyboard, background chatter, whatever's realistically present when you'd use Sherlock. Save it under `./data/my-room-noise/` and reference it in `augmentation.background_paths` as shown in Section 4.

6. Pipeline Execution

Step 1: Setup

Download base data, noise profiles, and pre-computed embeddings:

```
bash  
  
livekit-wakeword setup sherlock.yaml
```

Step 2: Data Generation + Training + Export

Run synthetic data generation, augmentation (now including your room noise), and training:

```
bash  
  
livekit-wakeword run sherlock.yaml
```

If you're manually injecting your own voice recordings via the `.npy` fallback (Section 5a), do that step **between** `setup` and `run`, so your positive samples are present before training starts.

Step 3: Evaluation

Test performance against validation data:

```
bash  
  
livekit-wakeword eval sherlock.yaml
```

Check `output/sherlock/metrics.json` afterward:

- Look at the `"note"` field — `"optimal_threshold"` means the threshold optimizer succeeded within your `target_fp_per_hour` target. If it instead fell back to maximizing balanced accuracy, that's a signal the model needs more/better data before you deploy it, not just a threshold tweak.
- Confirm recall is comfortably above your 70% bar.

Step 4: Sanity-check with your own held-out clips

Record a handful of fresh "Sherlock" clips you did **not** include in training, and run them through the exported model manually before wiring it into `session_manager.py` — this catches overfitting to your training set that the automated eval might not fully surface.

7. Integration & Deployment

Once exported to `./output/sherlock/sherlock.onnx`:

```
python
```

```
import asyncio
from livekit.wakeword import WakeWordModel, WakeWordListener

model = WakeWordModel(models=["./output/sherlock/sherlock.onnx"])

async def main():
    async with WakeWordListener(model, threshold=0.5, debounce=2.0) as listener:
        print("Listening for 'Sherlock' variations...")
        while True:
            detection = await listener.wait_for_detection()
            print(f"Detected {detection.name}! (confidence={detection.confidence})")

asyncio.run(main())
```

Note: this is the piece that plugs into the `WakeWordDetector.process(pcm_chunk)` interface expected by `session_manager.py` — wrap `WakeWordListener`'s detection loop so `process()` returns `True` once a detection with sufficient confidence comes through.

8. Summary of Changes from v1

Change	Why
Corrected CLI syntax (positional args, not <code>--config</code> flag)	Matches current documented usage
Added GPU/CUDA verification step	Confirms your RTX 3050 is actually being used, not silently falling back to CPU
Added <code>--help</code> check before running	Command names have shifted across versions — cheap insurance
Added own-voice recording + injection (Section 5a)	Closes the synthetic-vs-real accuracy gap discussed earlier
Added own-room background noise (Section 5b)	Documented, low-effort way to cut false positives specific to your environment
Added held-out sanity check (Step 4)	Catches overfitting the automated eval might miss